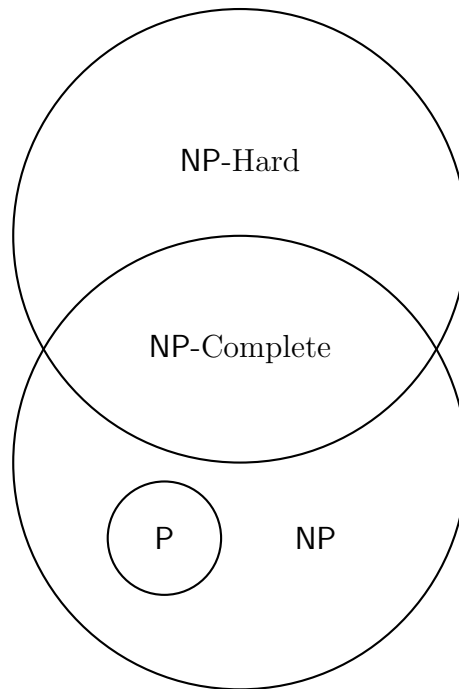


Practice Exam 4: Intro to Complexity

Abraham Ladha

- This is the CS 3510 practice exam for exam 4.
- Topics include: Algorithmic Complexity
- The number of questions on this practice exam is roughly what you could expect on the real exam.



1. For certain, this graph shows an accurate depiction of the relationship between the sets P, NP, NP-Hard, and NP-Complete (this is a trick question).

☐ True
☒ **False**
☒ **Maybe**

Frankly I'd say false because this question is essentially asking, does $P \neq NP$ and as a matter of fact, we do not know. However maybe is also a correct answer as well to provide more leniency.

2. If a quantum researcher discovers a way to run infinitely many polynomial-time algorithms at one time, then for quantum computers

☐ They have proven $P = NP$
☒ **They have not proven $P = NP$**

No we don't know enough about the machine the researcher is running on. We do not know if they have found an algorithm or a nondeterministic machine. If they have found a nondeterministic machine, that says very little because we already know $NP = NP$.

3. You've just made a break through and found a polynomial solution to the **LimitSAT**! The issue is that it's not exactly the same as normal **SAT** because each variable can only appear in at most three clauses, and there can only be at most three literals per clauses. Can you show that this is **NP-Complete**, thus showing $P = NP$ (Obviously this is a made up scenario because we don't actually know if $P = NP$, but nonetheless, show that **LimitSAT** is in **NP-Complete**)?

Solution:

First we'll prove that this is in the class **NP**. To do this, we could take the assignments as a witness and check that it evaluates to true like a standard **SAT** problem.

Now we must show that the problem is in the class **NP-hard**; we will reduce **3SAT** with exactly three clauses to **LimitSAT**. In our transformation we'll take every instance of a variable which appears more than once and replace it with some element in a set of related variables; for instance if we were to replace instances of x , we'd replace it with $x_0, x_1, \dots, x_n$. We need to create implications between the clauses in order to make them all equal, so we'll add $(\bar{x}_1 \vee x_2) \wedge (\bar{x}_2 \vee x_3) \wedge \dots \wedge (\bar{x}_n \vee x_1)$

This is trivially correct because if there's a satisfying assignment for normal **3SAT**, then the same must exist for **LimitSAT**. If there's not a satisfying assignment for **3SAT**, then there must not be one for **LimitSAT**. This is because all the variables corresponding to the original variables are equal by the implications, so this will map correctly, meaning the **LimitSAT** problem is in **NP-complete**.

4. As a student, you're trying to determine your schedule for the upcoming semester. For a particular day, there's some set of classes $s \in S$, each class has some meeting times (a class can meet multiple times in the day i.e. lecture and recitation). You need to determine whether you can schedule k non-overlapping classes throughout the day. Show this problem is **NP-Complete**.

Solution:

We can verify that a witness is valid by simply checking that there are no overlapping classes.

We can show that this problem is **NP-hard** by doing a reduction from independent set. We will take every node in independent set as a class and take the edges as individual meeting times for that class. Each node which shares an edge will have an overlapping time. This will not allow two nodes that share an edge to be chosen together. We will map k to k in independent set.

This is correct because it correctly maps the rules from independent set to the scheduling problem. If a problem is valid in independent set, the classes corresponding to the chosen nodes can also be chosen because they do not have overlapping times. If a problem is invalid in independent set, no more classes than the number for independent set can be chosen because the possible sets of classes which can be chosen is the same as the possible sets of nodes. Then this problem is in **NP-complete**.

5. In **Daniel's-Two-Sum** problem, you're given a set of integers S , and you attempt to split the set up into two non-overlapping sets A and B such that $A \cup B = S$, $A \cap B = \emptyset$ and

$$\sum_{a \in A} a = \sum_{b \in B} b$$

Decide whether **Daniel's-Two-Sum** is NP-Complete and give sufficient evidence including either a proof it's NP-Complete or a polynomial time algorithm to solve it.

Solution: This problem is in NP because we can do a verification by checking the equality the two of the sets in polynomial time.

We will reduce from SUBSETSUM. Given a set S and target t for SUBSETSUM, we will create the set $S' = S \cup s - 2t$ and pass that into **Daniel's-Two-Sum**.

If there's a solution to SUBSETSUM, this implies there's some set of numbers which add to t . The sum of the other numbers will be $s - t$; since we added $s - 2t$, if we add this number to the side that sums to t , it will equal $s - t$, meaning there are two equal sized sets. If there's no valid solution to SUBSETSUM, this proof will not work. Then we know it's NP-complete.

6. Show that the **Subgraph-Isomorphism** is in the class NP-complete. The problem is as follows: given as input two undirected graphs G and H , determine whether G is a subgraph of H .

Solution: We know this problems in NP by using the witness as the set of chosen nodes in the H which is equal to G . The witness is not only the set of nodes, but the order they are in. It specifies exactly where the subgraph is.

We will reduce from Clique. Note that if a graph has a clique, thats really just a complete subgraph. Given a graph for clique and a size c , construct a new complete graph with c nodes. Plug the provided graph into H and the constructed graph into G .

If there is a clique in the original graph the **Subgraph Isomorphism** problem must find it or decide it can't because the clique has been passed in. It can not find another clique because that would imply it had found a subgraph not present in the actual graph. If there is not a clique in the original graph, it could not find say there is one because that would imply that a clique does exist even though there isn't.

7. The Frog Software Foundation has recently come out with Knapsack as a service, allowing you to solve the Knapsack decision problem, wherein you input your knapsack problem and a value, and the service will tell you if the best value in Knapsack is above the provided value. How could you use this to efficiently solve the Knapsack optimization problem wherein you're given the Knapsack parameters and you want to find the best possible value. Then discuss your algorithms runtime.

Solution: You're trying to find the maximum number while still being able to satisfy Knapsack. Luckily since the problem is formulated by giving a lower bound, the function provided by the Frog Software Foundation is Monotonic Decreasing with respect to a given lower bound. This means you could just count up until you find a lower bound which does not work, but this will take exponential time with respect to the number of bits. Luckily, we know that we can do a binary search on a monotonic function in logarithmic time. This means we can find the highest lower bound in linear time.

For example, for some knapsack instance, fix the items and capacity and try values $V = 1, 10, 100, 1000, 10000 \dots$ in binary. The Frogs will say "higher, higher, higher, higher, lower". When they say lower, you know the first bit of V , and its length. Then for the rest of the bits you infer them one at a time. For example if you try 110000, and the Frogs say lower, you know it is at most 101111, or that that second bit was a zero. If it said higher, then you know it was at most 111111, or that the second bit was a one. You proceed bit by bit to reconstruct the maximum value V . This takes $\log V$ steps.